Portfolio Publisher

Full Project Documentation — File-by-File Technical Report

Project Documentation

August 2026

Contents

1. Executive Summary

Portfolio Publisher is a full-stack, multi-user web application that lets anyone create an account and instantly get their own live, shareable online portfolio (for example: `portfoliopublisher.vercel.app/u/jane-doe`). Instead of a developer having to build a personal website from scratch, a user signs up, fills in their details through an admin dashboard (profile, education, experience, projects, skills, certificates, achievements, activities), and the system automatically publishes a public, professional portfolio page — complete with a shareable link and a downloadable QR code for resumes, LinkedIn, or business cards.

Why this project matters (in plain terms):

- It solves a real problem — students and professionals need an online portfolio but most don't want to write HTML/CSS or pay for a website builder.
- Every account is fully isolated and secure: one person's data can never be seen or edited by another, enforced on the server, not just hidden in the interface.
- It includes production-grade features that go beyond a typical student project: email verification, password reset, rate limiting, analytics, a built-in AI chat assistant that can answer visitor questions about the portfolio owner, 10 selectable colour themes, animated 3D backgrounds, and full deployment automation via Docker.

Technology stack at a glance:

Layer	Technology	Purpose
Frontend	React 18 + Vite + Tailwind CSS	The user interface — what visitors and account owners see and click
Backend	Node.js + Express.js	The API server — handles logins, saves data, enforces security
Database	MongoDB + Mongoose	Stores all accounts and portfolio content
Authentication	JWT (JSON Web Tokens) + refresh cookies	Keeps accounts secure and sessions logged in
QR Codes	`qrcode` library	Generates a scannable code for each user's portfolio link
Deployment	Docker / Vercel (frontend) / Render (backend) / MongoDB Atlas (database)	How the finished app is hosted and made publicly available

This report walks through **every file and folder** in the project, explains **what it does**, and — importantly — **why it exists**, so that this documentation can be used both as a personal reference and as a document to hand to managers, reviewers, or non-technical stakeholders.

---

2. How the Project Is Organized (Big Picture)

The project is split into two independent applications that talk to each other over the internet:

```
portfoliopublisher-main/
├── frontend/     ← The website itself (what users see in their browser)
├── backend/      ← The server/API (handles data, logins, security)
├── docs/         ← Written documentation (architecture, API reference, deployment)
├── docker-compose.yml   ← Runs the whole system with one command
└── README.md, SECURITY.md, THEME_SYSTEM.md, etc. ← Project-level notes
```

Analogy: think of the **frontend** as a restaurant's dining room and menu (what the customer sees and interacts with), and the **backend** as the kitchen (where the actual work happens — checking IDs, preparing orders, storing inventory). The **database** is the pantry where all the ingredients (user data) are kept safely.

---

3. Root-Level Files

File	Purpose & Why It Exists
`README.md`	The main entry-point document. Explains what the app does, the tech stack, folder layout, quick-start setup instructions, the full API reference, deployment steps, and environment variables. This is the first thing any new developer or reviewer reads.

File	Purpose & Why It Exists
`SECURITY.md`	States which versions of the app receive security updates and how to report a vulnerability — standard open-source practice that signals the project takes security seriously.
`THEME_SYSTEM.md`	Documents the 10-colour theme engine in detail — which files control it and how the whole site re-colours instantly when a user changes theme.
`BATCH_A_CHANGES.md`	A changelog describing a specific round of upgrades: authentication hardening, OTP (one-time password) email verification, forgot-password flow, and general security improvements. Useful for tracking what was improved and when.
`FINAL_DELIVERABLES.md`	An index mapping every planned project "phase" (17 in total, e.g. audit, recruiter UX, auth upgrade, theme system, 3D backgrounds, AI assistant, performance, SEO, security, analytics, deliverables) to where that work lives in the codebase. This is effectively a project completion checklist — very useful to show a manager exactly what was delivered.
`docker-compose.yml`	A single configuration file that starts the **entire system** (frontend + backend + MongoDB database) with one command (`docker compose up`). This removes the need to manually install Node.js, MongoDB, etc. on every machine — it packages the whole app so it runs identically anywhere.
`package.json` (root)	A minimal top-level manifest, mostly used for any root-level scripts/tooling that apply across both `frontend/` and `backend/`.

---

4. Documentation Folder (`docs/`)

File	Purpose & Why It Exists
`docs/ARCHITECTURE.md`	The technical blueprint of the system: full folder structure, the database schema (what fields each data collection stores), and diagrams of how signup/login, password reset, and the AI assistant work step by step. This is the file to read to understand *how the pieces fit together*.

File	Purpose & Why It Exists
`docs/API.md`	A complete reference of every API endpoint the backend exposes (URL, HTTP method, whether login is required, what it does). This is what a frontend developer — or an external integrator — would use to know exactly how to talk to the server.
`docs/DEPLOYMENT.md`	Step-by-step instructions for putting the app online, covering two options: (A) one-command Docker deployment, or (B) deploying the frontend to Vercel, backend to Render, and database to MongoDB Atlas. Also covers setting up outgoing email and the automated testing pipeline (CI/CD).

---

5. Backend (`backend/`) — The Server

The backend is a Node.js/Express application. Its job is to be the single source of truth: it checks who is logged in, makes sure users can only see/edit their own data, saves everything to the database, sends emails, and generates QR codes.

5.1 Entry Point & Configuration

File	Purpose
`server.js`	The application's starting point. Wires together all the security middleware (Helmet for HTTP security headers, CORS, cookie parsing, protection against malicious database queries, response compression), connects to MongoDB, and mounts every route group (`/api/auth`, `/api/profile`, `/api/projects`, etc.). Running `node server.js` is what actually starts the API server.
`config/db.js`	A single, reusable function that opens the connection to the MongoDB database. Centralising this in one file (instead of repeating the connection code elsewhere) avoids bugs from duplicated logic.
`package.json` / `package-lock.json`	Lists every backend dependency (Express, Mongoose, JWT libraries, email libraries, etc.) and locks their exact versions so the app behaves identically on every machine it's installed on.
`Dockerfile`	Instructions for packaging the backend into a Docker container — a self-contained, portable unit that can run on any server without needing Node.js pre-installed.

File	Purpose
`vercel.json`	Configuration used if the backend is deployed on Vercel instead of Render.

5.2 Models (`backend/models/`) — The Data Blueprints

Each file here defines the *shape* of one type of data stored in MongoDB (Mongoose "schemas"). Think of each model as a spreadsheet template that says exactly what columns exist and what rules apply.

File	Purpose
`User.js`	The core account model: username, email, hashed password, verification status, role, and a list of active login sessions (refresh tokens). Also enforces the rules for valid usernames (3–30 characters, lowercase letters/numbers/hyphens) and strong passwords (uppercase, lowercase, number, special character), and blocks reserved words like "admin" or "login" from being used as a username, since those would clash with the app's own URLs.
`Profile.js`	One-to-one with a user: their name, professional title, "about me" text, contact info, profile picture, résumé file, and social media links — the core content shown at the top of a portfolio.
`Education.js`, `Experience.js`, `Projects.js`, `Skills.js`, `Achievements.js`, `Activities.js`, `Certificate.js`	Each is a repeatable content section a user can add multiple entries to (e.g. several jobs under Experience, several projects under Projects). Every entry is tagged with a `user` reference, which is how the system guarantees one user's items never mix with another's.
`WhyHire.js`	Stores the content for a "Why hire me" section aimed specifically at recruiters — a value-proposition summary.
`Otp.js`	Temporary one-time-password records used for email verification and password resets. Codes are stored **hashed**, expire automatically after 5 minutes, and track failed attempts to prevent guessing.
`LoginActivity.js`	An audit log of login events (success/failure, IP address, device, browser) — used to power a "recent activity" view and to detect logins from new devices.
`Visit.js`	Records each time someone views a public portfolio page — powers visitor analytics.
`ResumeDownload.js`	Records every time a visitor downloads a user's résumé — another analytics data point.

File	Purpose
`ContactMessage.js`	Stores messages sent through a portfolio's public "contact me" form, along with a read/unread flag.
`ThemeEvent.js`	Records which colour theme visitors pick — used for analytics on theme popularity.

5.3 Controllers (`backend/controllers/`) — The Business Logic

Controllers contain the actual logic that runs when a request hits an endpoint — the "kitchen staff" who do the real work.

File	Purpose
`authController.js`	Handles registration, email verification via OTP, login, logout, token refresh, and the full forgot-password flow. This is the largest and most security-critical controller.
`crudController.js`	A **generic, reusable** Create/Read/Update/Delete factory. Rather than writing near-identical code six times for Education, Experience, Projects, Skills, Achievements and Activities, this one file generates all the standard operations for any content type, always scoped to the correct user. This is a good example of avoiding repeated ("copy-pasted") code.
`profileController.js`	Manages reading and updating a user's profile, including handling the profile picture and résumé file upload, and assembling everything needed to render a public portfolio page.
`portfolioController.js`	Generates each user's shareable portfolio link and the PNG QR code that points to it.
`analyticsController.js`	Records portfolio page visits, résumé downloads, and theme selections, and — for the logged-in owner — compiles this into dashboard statistics (visitor counts, device/browser breakdown, referrers, and recent activity).
`assistantController.js`	Powers the AI chat widget shown on each public portfolio. It gathers a portfolio owner's real data (profile, skills, projects, experience, education, achievements) into a compact "knowledge base" so the assistant can only ever answer using that person's real facts — it cannot invent information or leak anything outside that portfolio.

5.4 Middleware (`backend/middleware/`) — The Security Checkpoints

Middleware are functions that run *before* a controller, acting like security guards or receptionists that check a request before it's allowed through.

File	Purpose
`auth.js`	Verifies the login token (JWT) sent with a request, confirms the user still exists, and rejects tokens issued before the user's last password change (so changing your password instantly logs out old sessions everywhere).
`resolveUser.js`	Converts a `:username` in a public portfolio URL (like `/u/jane-doe`) into a real account. If the username doesn't exist, it returns a clean "not found" response instead of a confusing empty page.
`validate.js`	Runs input-validation rules and, if anything is wrong (e.g. an invalid email format), returns a clear error message the frontend can display.
`rateLimiters.js`	Limits how many times an action (like login attempts or OTP requests) can be performed in a given time window, to block brute-force attacks and abuse.
`lockout.js`	Tracks failed login attempts and applies a progressively increasing delay (and eventually a temporary lockout) after repeated failures — an extra layer of brute-force protection beyond rate limiting.
`upload.js`	Handles image and résumé/PDF file uploads (using Multer), optionally storing them on Cloudinary (cloud storage) when configured, or on local disk otherwise.

5.5 Routes (`backend/routes/`) — The URL Map

Each route file defines the actual URLs (`/api/...`) available for one resource and which controller function/middleware handles each one.

File	Purpose
`auth.js`	Routes for register, login, logout, verify-email, forgot/reset password, token refresh.
`profile.js`	Routes for reading/updating the logged-in user's profile and viewing any user's public profile.
`education.js`, `experience.js`, `projects.js`, `skills.js`, `achievements.js`, `activities.js`, `certificates.js`	Each follows the same consistent pattern: get my items, get another user's public items, create, update, delete — all built on the shared `crudController`.
`portfolio.js`	Routes for the shareable link, the QR code image, recording visits/theme choices/résumé downloads, the public contact form, the AI assistant chat endpoint, and the owner-only analytics/messages endpoints.

5.6 Utilities (`backend/utils/`) — Shared Helper Functions

File	Purpose
`token.js`	Creates and manages the short-lived login token (15 minutes) and the longer-lived refresh token (7–30 days) that keeps users signed in without repeatedly asking for a password.
`otp.js`	Generates secure 6-digit one-time codes, hashes them before saving (so even a database leak can't reveal valid codes), and enforces expiry/attempt/cooldown rules.
`email.js`	Sends all outgoing emails (verification codes, password reset codes, login alerts) using styled, responsive email templates. If no email service is configured, it safely falls back to printing the code to the server log instead of failing — useful for local development and testing.
`device.js`	A lightweight tool that reads a browser's "user agent" string to figure out the operating system and browser being used — used for security alerts like "new device logged in" and for analytics.

5.7 Tests (`backend/tests/`)

File	Purpose
`auth.flow.test.js`	An automated, end-to-end test that walks through the entire signup → email verification → login flow against a temporary in-memory database, confirming the authentication system actually works correctly.
`api.flow.test.js`	An automated test covering the analytics, contact form, and AI assistant endpoints.

Automated tests like these matter because they let the app be verified quickly (and automatically, via CI) after every change, without a person having to manually click through every feature by hand each time.

---

6. Frontend (`frontend/`) — The Website

The frontend is a React application built with Vite (a fast build tool) and styled with Tailwind CSS (a utility-based styling system). This is everything the user actually sees and clicks.

6.1 Entry Point & Configuration

File	Purpose
`src/main.jsx`	The very first file that runs — it mounts the whole React application into the webpage.
`src/App.jsx`	Defines every page/route in the app (e.g. `/`, `/u/:username`, `/admin/login`, `/admin/dashboard/...`) and wraps them in the app's global providers (theme, background, authentication). Pages are **lazy-loaded**, meaning the code for a page is only downloaded when a visitor actually navigates to it — this keeps the initial page load fast.
`index.html`	The single HTML page that hosts the React app (standard for a "Single Page Application").
`vite.config.js`	Configuration for the Vite build tool (how the app is bundled for development and production).
`tailwind.config.js`	Configures the Tailwind styling system, including mapping colour names like "primary" to the CSS variables that drive the 10-theme colour system.
`postcss.config.js`	Supporting configuration for CSS processing, required by Tailwind.
`nginx.conf`	Web-server configuration used when the frontend is deployed via Docker — serves the built site and forwards `/api` requests to the backend.
`Dockerfile`	Packages the frontend into a Docker container for deployment.
`vercel.json`	Deployment configuration for hosting the frontend on Vercel, including the rule that redirects all page routes back to `index.html` (required for single-page apps).
`package.json` / `package-lock.json`	Lists all frontend libraries (React, Tailwind, Framer Motion for animation, Three.js for 3D graphics, Axios for API calls, etc.) with locked versions.

6.2 Pages (`src/pages/`) — The Screens

File	Purpose
`Landing.jsx`	The public homepage of the *platform itself* (not a user's portfolio) — introduces the product and invites visitors to sign up.

File	Purpose
`PortfolioLayout.jsx`	The wrapper for every published portfolio at `/u/:username`. It looks up the username, shows a loading state while fetching data, and displays a friendly "portfolio not found" or "still under construction" page if appropriate, instead of a broken-looking blank page.
`Home.jsx`	The main content of a published portfolio — hero section, skills, "why hire me", stats, and contact form, assembled from that user's data.
`Education.jsx`, `Experience.jsx`, `Projects.jsx`, `Certificates.jsx`, `Activities.jsx`	Individual sections/pages of a public portfolio, each displaying one category of content. `Projects.jsx` additionally includes search and filter-by-technology functionality.
`NotFound.jsx`	A general 404 "page not found" screen for any invalid URL.
`Privacy.jsx`, `Terms.jsx`	Standard privacy policy and terms-of-service pages.
`pages/admin/Login.jsx`, `Signup.jsx`	Account login and account creation screens.
`pages/admin/VerifyEmail.jsx`	The screen where a new user enters the 6-digit code emailed to them to activate their account.
`pages/admin/ForgotPassword.jsx`, `ResetPassword.jsx`	The two-step password-recovery flow (request a reset code, then set a new password).
`pages/admin/Dashboard.jsx`	The main control panel after logging in — a sidebar linking to every editable section (Profile, Share, Education, Experience, Projects, Skills, Why Hire Me, Achievements, Activities).
`pages/admin/Profile.jsx`	The form where a user edits their name, title, bio, contact details, profile photo, and résumé.
`pages/admin/Sections.jsx`	The reusable screen used for managing list-style content sections (Education, Experience, etc.) inside the dashboard.
`pages/admin/Share.jsx`	Displays the user's permanent portfolio link with a one-click copy button, plus their downloadable QR code — the "get the word out" screen.

6.3 Components (`src/components/`) — Reusable Building Blocks

`components/common/` — small, reusable pieces used throughout the app:

File	Purpose
`AssistantWidget.jsx`	The floating AI chat button/window shown on every public portfolio, letting visitors ask questions like “what are their top skills?” and get answers generated from that person’s real portfolio data.
`BackgroundCustomizer.jsx`	A settings panel letting a portfolio owner pick and fine-tune (speed, density, glow) one of the animated 3D backgrounds.
`OtpInput.jsx`	A user-friendly 6-box code entry field (used on the verify-email and reset-password screens) that supports pasting a code and auto-advances between boxes.
`PasswordStrength.jsx`	Live visual feedback showing whether a typed password meets all the strength requirements (matches the same rules enforced on the backend, so the user sees the same rule in both places).
`ProtectedRoute.jsx`	Guards admin/dashboard pages so only a logged-in, verified user can reach them; anyone else is redirected to the login page.
`Section.jsx`	A reusable wrapper that adds a “fade/slide in as you scroll” animation to any section of content.
`Seo.jsx`	Manages the page title, meta description, and social-media preview tags (Open Graph/Twitter cards) for each page — improves how the site looks when shared and how it’s found on search engines.
`Spinner.jsx`	The loading spinner shown while data is being fetched.
`ThemeSwitcher.jsx`	The dropdown/panel that lets a user switch between light/dark mode and the 10 colour palettes, with a live preview of each.
`Tilt.jsx`	A lightweight hover effect that gives cards (like project cards) a subtle 3D tilt as the mouse moves over them, for visual polish.

`components/admin/` — pieces specific to the logged-in dashboard:

File	Purpose
`AdminCRUD.jsx`	A single, generic “add/edit/delete” table-and-form component reused across Education, Experience, Projects, Skills, Achievements and Activities — so each of those admin screens didn’t need its own custom form code.
`AuthShell.jsx`	The shared visual frame (card, background, theming) used by every authentication screen (login, signup, verify, forgot/reset password) so they look and behave consistently.

components/landing/ — pieces used only on the platform's own marketing homepage:

File	Purpose
`Navbar.jsx`, `FooterSection.jsx`, `FooterModal.jsx`	The landing page's navigation bar and footer (with pop-up modals for things like privacy/terms).
`HeroSection.jsx`	The large introductory banner at the top of the landing page.
`FeaturesSection.jsx`	A section listing the product's key features.
`CTASection.jsx`	A "Call To Action" section encouraging visitors to sign up.

components/layout/ — pieces used on every *published portfolio* (not the landing page):

File	Purpose
`Navbar.jsx`	The navigation bar shown on a user's public portfolio, aware of whose portfolio is being viewed.
`Footer.jsx`	The footer shown on a user's public portfolio.

components/three/ — the animated backgrounds:

File	Purpose
`ThreeScene.jsx`	Renders the actual WebGL 3D scenes (particles, neural network, galaxy, grid, spheres, waves) using Three.js.
`SceneBackground.jsx`	Decides which background to show, lazy-loads the 3D engine only when needed, and automatically falls back to a simple CSS gradient on low-end devices or when the visitor's system prefers reduced motion (an accessibility consideration).

6.4 Context (`src/context/`) — App-Wide State

React "context" files share information across the entire app without having to manually pass it into every component.

File	Purpose
`AuthContext.jsx`	Tracks whether someone is logged in, who they are, and handles silently restoring a session on page reload using the refresh cookie.
`ThemeContext.jsx`	Tracks the current light/dark mode and which of the 10 colour palettes is active, saves the choice so it persists between visits, and applies it across the whole site instantly.

File	Purpose
`BackgroundContext.jsx`	Tracks which animated 3D background is selected and its custom settings (speed, density, glow, etc.).

6.5 Hooks (`src/hooks/`) — Reusable Logic

File	Purpose
`useTypewriter.js`	Produces the animated "typing" text effect used in the hero section.
`useScrollAnimation.js`	Reusable logic for triggering animations as elements scroll into view.
`useMousePosition.js`	Tracks the mouse's position on screen — used for interactive hover/tilt effects.

6.6 Services (`src/services/`)

File	Purpose
`api.js`	The single place where the frontend talks to the backend (using Axios). It keeps the short-lived login token safely in memory (not in the browser's local storage, which is more vulnerable to attacks) and automatically refreshes it using the secure cookie when it expires. Every API call used throughout the app (profile, education, projects, portfolio, analytics, etc.) is defined here once, so the rest of the app doesn't repeat network-request code.

6.7 Styling

File	Purpose
`src/index.css`	Defines the 10 colour palettes as reusable CSS variables and the global styling rules that let the entire site re-colour instantly when the theme changes, without editing individual components.

---

7. Key Concepts Explained Simply

Multi-tenancy (multi-user isolation): Every piece of content in the database (a project, a skill, a job entry) is stamped with a reference to exactly one user. Every single database query on the backend filters by that reference. This is enforced on the server — not just hidden in

the interface — so it's not possible for one account to view or modify another account's data, even by guessing an ID.

JWT + refresh token authentication: When someone logs in, they get two things: a short-lived "access token" (15 minutes) used to make requests, and a longer-lived "refresh token" stored in a secure cookie that the browser can't read with JavaScript. When the access token expires, the app quietly asks for a new one using the refresh token, so the user stays logged in without noticing. This limits how long a stolen access token could ever be useful.

OTP (One-Time Password) email verification: Instead of an account being usable immediately, new users must enter a 6-digit code emailed to them. This confirms the email address is real and reachable, which protects both the platform (fewer fake/spam accounts) and the user (their portfolio contact form actually reaches them).

Rate limiting & lockout: Limits are placed on how often sensitive actions (login, password reset requests) can be attempted, to block automated attacks that try to guess passwords or spam the system.

The AI Assistant: A visitor to someone's portfolio can chat with an assistant that only knows facts pulled directly from that portfolio's real data (skills, projects, experience, etc.). If an AI provider key is configured it uses that; if not, it automatically falls back to a built-in rule-based mode — so the feature always works, with no required paid service.

Theming engine: Ten complete colour palettes are defined once as CSS variables. Every existing button, badge, and text colour in the app already references those variables, so switching a theme recolours the entire site instantly, without needing to update each component individually.

Analytics: The platform quietly records portfolio page views, résumé downloads, and theme choices (with device/browser/referrer info), and shows a summary of this activity back to the account owner in their dashboard — similar to a simplified Google Analytics, specific to their own portfolio.

---

8. Summary for Managers / Non-Technical Stakeholders

In short, this project is a working, secure, multi-user portfolio-building platform, comparable in scope to a lightweight version of a product like Carrd or Notion-for-portfolios, but purpose-built for showcasing a person's education, work experience, and projects to recruiters.

What was built: - A public marketing/landing page and a self-service sign-up flow - A secure account system with email verification and password recovery - A private dashboard where each user manages their own portfolio content - An automatically generated, permanent public portfolio page per user, with a shareable link and QR code - Visitor analytics and a contact-message inbox for each user - An AI chat assistant on every portfolio - A polished, animated, themeable user interface - Automated tests and a one-command deployment setup (Docker), plus managed hosting instructions (Vercel/Render/MongoDB Atlas)

Why it's structured this way: the codebase separates the visual layer (frontend) from the data/security layer (backend), which is standard industry practice — it means the interface can be redesigned without touching how data is secured, and the API could power other clients (like a future mobile app) without changes.